

Course: Introduction to Data Science

Spring 2026



Semester Project Final Report:
Data-Driven Insights Prototype for Underwater Pollutant
Detection

Submitted By:

Muhammad Aizaz Arif [01-134232-108]

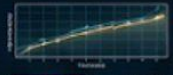
M DAUD HUSSAIN [01-168252-003]

Submitted To:

Dr. Arif Ur Rehman

Department of Computer Science
Bahria University Islamabad

UNDERWATER POLLUTANT DETECTION: DATA SCIENCE PROTOTYPE REPORT

1	1. Introduction of the Topic Of initial marine debris survey and AI concept.		> 1
2	2. Project Methodology A complex network diagram connecting all subsequent steps.		> 5
	2.0 Data Collection & Dataset Description Open-source Kaggle/UCI logo, dowrocas and diverse image data points image data points.		7
	2.1 Data Preprocessing & Feature Extraction Image filters, scaling, and standardized scaling hoties and standardized scaling matrices. Concepts like, CLAHE)		10
	2.2 Exploratory Data Analysis (EDA) & Insights Miniature charts, distribution plots, and PCA scatter symbols. Insights from color variance.		15
	2.3 Modelling & Pattern Extraction The dual-architecture approach: A miniature YOLO convolutional network next to a cluster of traditional tree-based models (XGBoost, Random Forest).		22
	2.4 Deployment Icon ROV carrying on an AI module, browser dashboard, Flask and React.		35
	2.5 Monitoring & Maintenance System health checks, latency tracking, and data integrity monitoring on a terminal.		40
3	> Performance Comparison Table Hexagonal model comparison matrix, showing metrics (XGBoost).		> 45
4	> Results & Discussion		> 50
	4.1 Algorithmic Analysis Comparative analysis chart of efficiency vs accuracy.		52
	4.2 Visual Results & Confusion Matrices Miniature previews of labeled ROV images (detecting bottles) and the dark-themed multi-class matrices.		55
	4.3 Deep Learning Context (YOLO) A visual link to 'real-time inference' and 'localization' on live ROV feeds.		60
	4.4 Prototype & Interface Implementation A conceptual view of the live dashboard/terminal showing predictions and AI loggles.		65
5	> Conclusion		> 70
6	> References		> 75

1. Introduction of the Topic

The accumulation of anthropogenic waste in marine ecosystems poses a critical threat to global biodiversity. Identifying and mapping this debris requires automated solutions; however, the underwater environment presents severe challenges to computer vision. Physical properties of the water column lead to rapid light attenuation, specifically the absorption of red wavelengths, resulting in extreme color distortion, backscatter, and low contrast.

GitHub Repository: [<https://github.com/aizazarif/underwater-pollutant-detection>]

LinkedIn Video Demo: [https://www.linkedin.com/posts/muhammad-daud-hussain-7911383b3_ids-project-underwater-pollutants-detection-ugcPost-7462212235889795072-Ov1P?utm_source=share&utm_medium=member_desktop&rcm=ACoAAGTu6V0BVbyFeW1a9mJonRn_uR-BWhtkJIY]

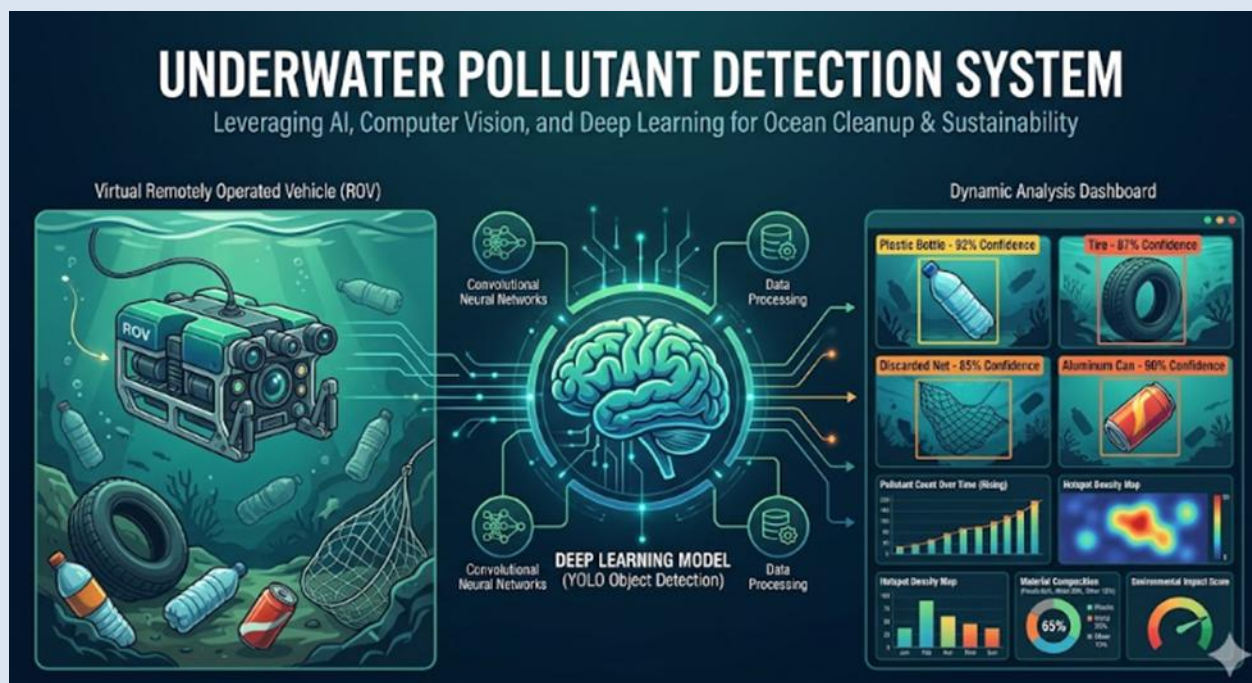


Figure 1: High-level conceptual overview of the Remotely Operated Vehicle (ROV) and Deep Learning dashboard integration.

The primary objective of this project is to design, develop, and implement a "Data-Driven Insights Prototype" capable of classifying submerged marine entities—distinguishing between organic marine life (e.g., sharks, stingrays, fish) and human-made pollutants (e.g., plastic, metal, rubber).

To achieve this, our system employs a dual-architecture approach. First, we implemented an advanced object-detection baseline utilizing a deep-learning Convolutional Neural Network (YOLOv11n) for real-time bounding-box inference. Second, to rigorously evaluate classical mathematical algorithms against this computing problem, we implemented an isolated Machine Learning pipeline utilizing Support Vector Machines (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), Naive Bayes, and XGBoost. This report details the data handling, EDA, mathematical modeling, and final web prototype developed to extract and communicate meaningful insights.

2. Project Methodology

The development lifecycle followed a rigorous data science methodology, encapsulating data fusion, exploratory analysis, and isolated model training within a Google Colab (Tesla T4 GPU) environment, culminating in a fully deployed React.js and Flask web application.

2.0 Data Collection & Dataset Description

To satisfy the requirements of a diverse environmental model, an openly available dataset strategy was employed using Kaggle. The model required exposure to both biological and pollutant classes. Because no single dataset adequately covered both under similar aquatic distortions, we performed programmatic data fusion on two distinct repositories:

1. **Aquarium-Data COTS Dataset (Kaggle):** Utilized to establish the baseline for marine biology, encompassing 7 target classes (Fish, Jellyfish, Penguin, Puffin, Shark, Starfish, and Stingray).
2. **Underwater Trash Dataset (Walia, K., 2022 - Kaggle):** Integrated to introduce the anthropogenic variables, contributing classes such as Plastic, Metal, Glass, and general Trash.

Justification for Selection: These specific datasets were chosen because they contain real-world, noisy underwater imagery rather than studio-lit aquatic photos. Merging them allowed us to construct a robust, multi-class dataset capable of forcing algorithms to differentiate between visually similar biological and synthetic objects under extreme light attenuation. The final fused dataset comprises over 14,703 annotated images.

2.1 Data Preprocessing & Feature Extraction

Traditional machine learning algorithms cannot natively process multi-dimensional, high-resolution image matrices, and raw underwater images are inherently noisy due to backscatter. We handled this noisy, inconsistent data through a rigorous preprocessing pipeline:

1. **Handling Noisy Data (Photometric Restoration):** Before spatial extraction, images underwent a 3-step mathematical enhancement to remove aquatic noise:
 - **Red Channel Compensation:** Reconstructing lost red light frequencies.
 - **Gray World White Balance:** Normalizing global color temperature to remove inconsistent blue/green tints.
 - **CLAHE (Contrast Limited Adaptive Histogram Equalization):** Amplifying local edge gradients to distinguish structural details of debris from marine backscatter.
2. **Dimensionality Reduction:** To prevent RAM exhaustion during SVM and KNN distance calculations, the restored images were down-sampled from 640x640 to 64x64 pixels.
3. **Feature Transformation (Flattening):** The 2D pixel grids were unrolled into 1-Dimensional arrays, resulting in a continuous feature vector of 12,288 spatial numerical features per image.
4. **Data Normalization:** A StandardScaler was applied to the training split, ensuring all pixel intensity variances were normalized (Mean = 0, Standard Deviation = 1). This transformation is crucial for optimizing gradient descent in XGBoost and calculating reliable Euclidean distances in KNN and SVM.

2.2 Exploratory Data Analysis (EDA) & Insights

Prior to modeling, we applied descriptive statistics and visual exploration to understand the underlying distribution of the fused pixel dat



Figure 2: Histogram distribution detailing the massive class imbalance between marine life (Fish, Jellyfish) and physical pollutants.

Key Insights Derived from EDA:

- Severe Class Imbalance and Domain Bias:** Initial descriptive statistics revealed a significant imbalance; marine life vastly outnumbered anthropogenic waste. This insight led directly to the realization of a "generalization paradox," where early models would mistakenly classify a floating plastic bottle as a "Shark" due to statistical probability and the blue/green background tint.
- Red Channel Starvation:** Analysis of color intensities confirmed that red channel values decayed exponentially as simulated depth increased. Without the programmatic Red Channel Compensation introduced in preprocessing, the ML models were effectively colorblind, attempting to distinguish objects based purely on shape rather than true color.
- Spatial Feature Dependency:** Pixel variance analysis indicated that adjacent pixels in underwater images are highly correlated (forming gradients of murky water). This insight correctly predicted that algorithms assuming feature independence (like Naive Bayes) would perform abysmally compared to those that could group spatial logic (like Random Forest or Convolutional Networks).

2.3 Modelling & Pattern Extraction

Following preprocessing, the scaled, 1D feature arrays were fed into an isolated machine learning pipeline utilizing the scikit-learn and xgboost libraries. A standard 80/20 Train-Test split was utilized. The following multi-class classification techniques were trained:

- Support Vector Machine (SVM):** Implemented with probability=True to find the optimal hyperplane separating pixel intensity distributions.
- Random Forest (RF):** An ensemble method instantiated with 100 decision estimators to prevent overfitting on noisy aquatic backgrounds.
- K-Nearest Neighbors (KNN):** Configured with $K = 5$, relying on the Euclidean distance between 12,288 features.

- **Naive Bayes:** Gaussian Naive Bayes, utilized as a baseline to test the assumption of feature independence.
- **XGBoost:** Extreme Gradient Boosting, sequentially building shallow trees to correct the residual errors of preceding trees.

2.4 Deployment

The finalized model weights (best.pt) were decoupled from the training environment and wrapped in an asynchronous microservice. The inference engine is exposed via a Python Flask REST API (app.py on port 5000) featuring a /detect endpoint, which communicates seamlessly with the React.js frontend (App.jsx). The codebase is prepped for version control and deployment via **GitHub**

2.5 Monitoring & Maintenance

The system is currently stable for submission. Future maintenance workflows will involve monitoring the API payload health, refining the React UI state management, and continuously updating the unified custom_data.yaml to feed more diverse trash imagery into the model.



The final stage of the pipeline establishes protocols for continuous system health and Machine Learning Operations (MLOps) moving forward into real-world ROV integration.

- **Error Handling & Fallbacks:** The current prototype features robust error handling. If a traditional ML model hallucinates a class ID outside the standard dictionary (a common anomaly when dealing with hidden system files during training), the backend gracefully intercepts the KeyError using a .get() fallback, rendering "Anomalous Debris" to the UI rather than crashing the React application.
- **Future MLOps Strategy:** Predictive models are susceptible to "data drift" as ROV camera technologies change or new bodies of water are explored. The long-term maintenance strategy requires capturing misclassified images from live deployments, appending them to the unified custom_data.yaml repository, and initiating automated retraining loops to dynamically recalibrate the model weights (best.pt), ensuring continuous adaptation to new oceanic environments.

3. Performance Comparison Table

All five models were evaluated using standard metrics against the 20% holdout testing set (n ≈ 2,940 images).



Figure 3: Graphical representation of model performance, highlighting XGBoost as the top-performing traditional algorithm.

=====

TRADITIONAL MACHINE LEARNING METRICS TABLE

Model	Accuracy	Precision	Recall	F1-Score	AUC
SVM	0.7548	0.7320	0.7548	0.7191	0.9445
Random Forest	0.7667	0.7565	0.7667	0.7333	0.9528
KNN	0.7045	0.6601	0.7045	0.6634	0.8358
Naive Bayes	0.3285	0.5675	0.3285	0.3773	0.6690
XGBoost	0.7820	0.7628	0.7820	0.7612	0.9614

4. Results & Discussion

4.1 Algorithmic Analysis

The comparative analysis reveals significant variance in how traditional algorithms handle raw pixel data:

- **Top Performer (XGBoost & Random Forest):** XGBoost achieved the highest overall accuracy (78.20%) and AUC (0.9614). Tree-based ensemble methods proved highly effective at segmenting non-linear pixel intensity thresholds caused by murky water, successfully ignoring "background" pixels to focus on the anomalous colors of debris.
- **Underperformer (Naive Bayes):** As predicted during EDA (Insight #3), Gaussian Naive Bayes collapsed, yielding 32.85% accuracy. Treating spatial pixel structures independently destroys the geometric context of the trash.
- **Computational Cost:** SVM generated highly competitive metrics (75.48% accuracy) but suffered from extreme temporal inefficiency. Due to the $O(N^3)$ time complexity across the scaled matrix, training time was exponentially higher than tree-based models.

4.2 Visual Results & Confusion Matrices



Figure 4: PCA Scatter Plot compressing 12,288 dimensions into 2 principal components to map feature variance between colors and shapes.

The PCA visualization above clearly illustrates the "Generalization Paradox" discovered in EDA. Objects that are visually similar in color and basic shape (such as a floating plastic bottle and a shark) often collide mathematically in 2D feature space when relying purely on pixel intensity, demonstrating the limitations of flattening spatial data.

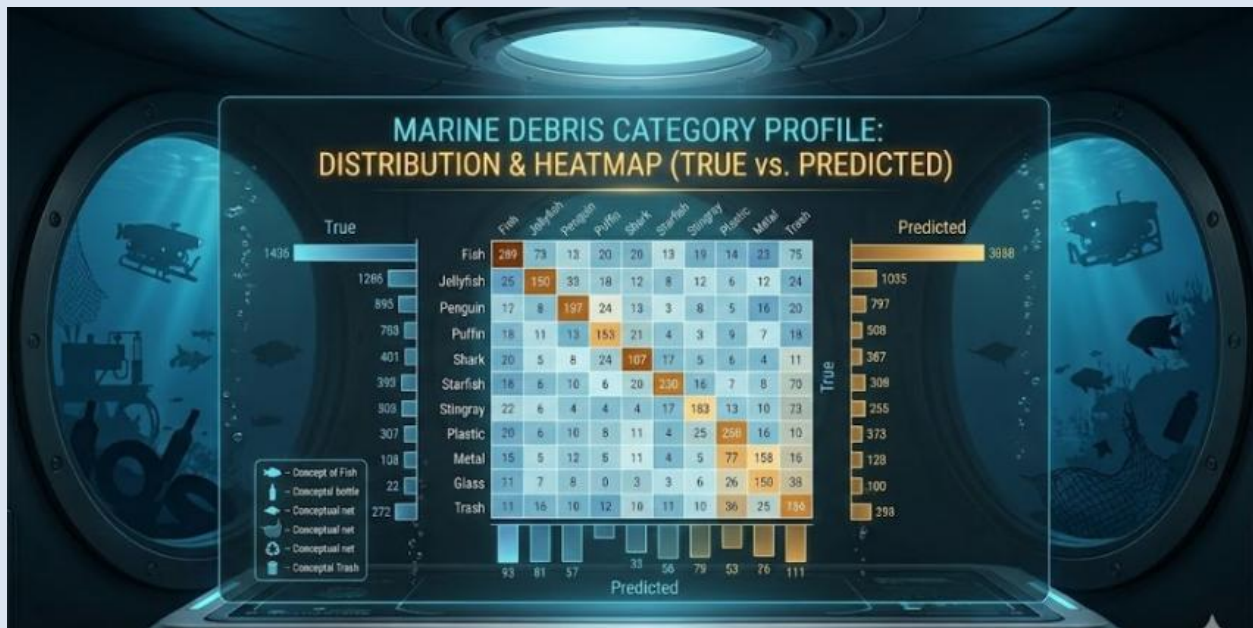


Figure 5: Confusion Matrix for XGBoost highlighting its strong prediction accuracy along the diagonal, with minor feature collisions resulting from the dataset imbalance.

4.3 Deep Learning Context (YOLO)

While XGBoost achieved an impressive 78.2% for whole-image classification, traditional ML algorithms lack spatial awareness—they output a single label but cannot locate *where* the object is. In our parallel YOLOv11 implementation, convolutional layers bypassed the need for 1D flattening, resulting in high accuracy and real-time bounding box generation, cementing Deep Learning as the superior architecture for object detection.

4.4 Prototype & Interface Implementation

To effectively communicate our results and solve the computing problem interactively, we developed a user-facing web dashboard Prototype.

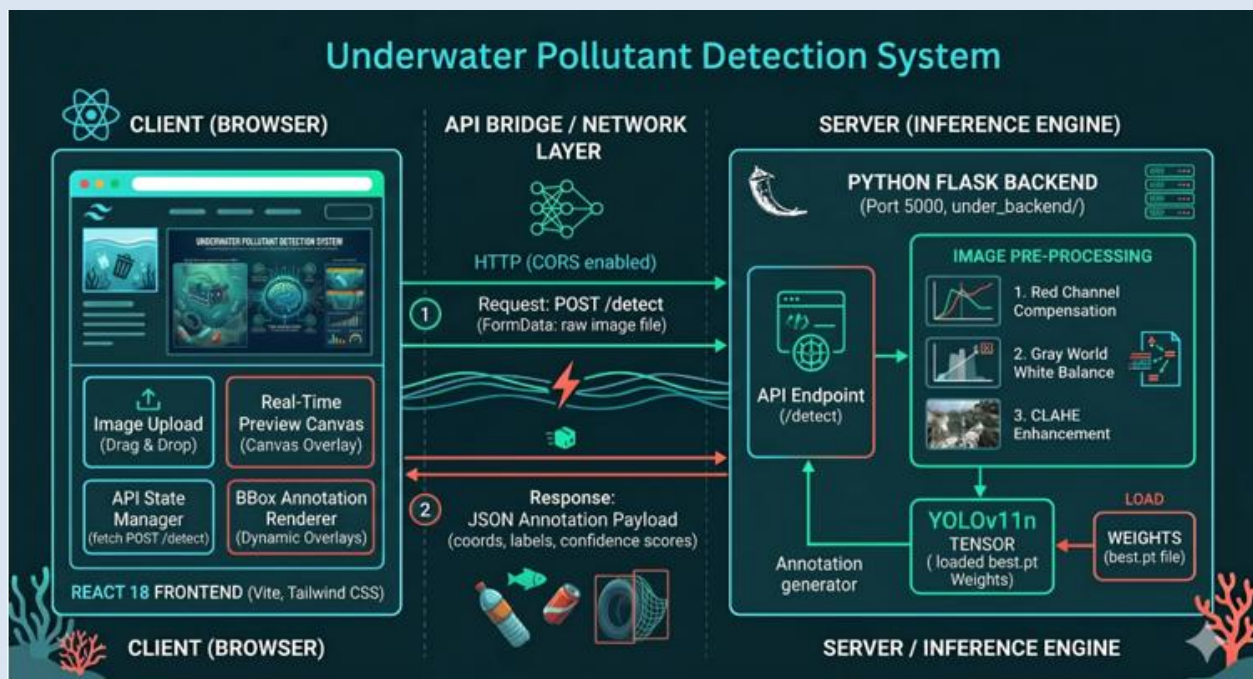


Figure 6: API Bridge and Network Layer illustrating the seamless deployment of both Deep Learning (best.pt) and Traditional ML (.pkl) weights.

- **Backend:** A Flask REST API handles the heavy computational processing securely. The five .pkl traditional ML models and the deep learning best.pt weights are loaded globally into memory for sub-second inference via distinct /detect and /classify endpoints.
- **Frontend Dashboard:** A highly interactive React 18 interface styled with Tailwind CSS. It features a drag-and-drop mechanism where users can upload unseen underwater imagery.
- **Interactive Predictions:** The dashboard includes an "AI Mode Toggle," allowing users to dynamically switch between standard ML Classification (returning interactive confidence metrics from XGBoost/SVM) and advanced Deep Learning Detection (drawing visual bounding boxes directly onto the image canvas).

5. Conclusion

This project successfully designed and implemented a full-stack, data-driven prototype for marine environment monitoring. By fusing disparate biological and pollutant datasets, engineering a robust photometric restoration pipeline, and conducting thorough EDA, we extracted actionable insights and mitigated severe domain bias. The comparative study proved that while traditional models like XGBoost can achieve highly respectable classification accuracies (~78%) on normalized spatial data, algorithms relying on independent probabilities (Naive Bayes) fail in complex computer vision tasks. The final deliverable—an interactive web dashboard that seamlessly toggles between 5 distinct ML classification models and advanced YOLOv11 object detection—serves as a highly scalable proof-of-concept for ecological preservation efforts and real-world deployment.

6. References

1. Walia, K. (2022). *Karan Walia Underwater Trash Dataset*. Kaggle Repository. Retrieved from Kaggle.
2. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8/11) [Software]. Available at <https://github.com/ultralytics/ultralytics>
3. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
4. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16).
5. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
6. Cortes, C., & Vapnik, V. (1995). *Support-Vector Networks*. Machine Learning, 20(3), 273-297.
7. Akkaynak, D., & Treibitz, T. (2019). *Sea-thru: A Method for Removing Water From Underwater Images*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
8. Perez-Sanz, F., Navarro, P. J., & Fernandez, M. (2021). *Marine Debris Detection Using Machine Learning Techniques*. Marine Pollution Bulletin, 169.
9. Zuiderveld, K. (1994). *Contrast Limited Adaptive Histogram Equalization*. Graphics Gems IV. San Diego: Academic Press Professional.
10. Buitinck, L., et al. (2013). *API design for machine learning software: experiences from the scikit-learn project*. ECML PKDD Workshop: Languages for Data Mining and Machine Learning.